ITA06020 MACHINE LEARNING

Experiment 4 – Artificial Neural Network Using Backpropagation

Aim:

To implement an Artificial Neural Network using the Backpropagation algorithm and test its performance using a real-world dataset.

Dataset: Breast Cancer Wisconsin Dataset (available in Scikit-learn).

Procedure

1. Import the required Python libraries.
2. Load the Breast Cancer dataset.
3. Split the dataset into training and testing data.
4. Normalize the input data.
5. Initialize the neural network weights.
6. Perform forward propagation and calculate the error.
7. Update weights using backpropagation.
8. Test the model and calculate accuracy.

Code:

```
import numpy as np

from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.metrics import accuracy_score

data = load_breast_cancer()

X = data.data

y = data.target.reshape(-1, 1)

X_train, X_test, y_train, y_test = train_test_split(

   X, y, test_size=0.2, random_state=42

)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)

np.random.seed(42)
```

```
W1 = np.random.randn(30, 8) * 0.1
W2 = np.random.randn(8, 1) * 0.1
b1 = np.zeros((1, 8))
b2 = np.zeros((1, 1))
def sigmoid(x):
    return 1 / (1 + np.exp(-np.clip(x, -500, 500)))
for epoch in range(1000):
    hidden = sigmoid(np.dot(X_train, W1) + b1)
    output = sigmoid(np.dot(hidden, W2) + b2)
    error = output - y_train
    dW2 = np.dot(hidden.T, error) / len(X_train)
    db2 = np.mean(error, axis=0, keepdims=True)
    dh = np.dot(error, W2.T) * hidden * (1 - hidden)
    dW1 = np.dot(X_train.T, dh) / len(X_train)
    db1 = np.mean(dh, axis=0, keepdims=True)
    W2 -= 0.1 * dW2
    b2 -= 0.1 * db2
    W1 -= 0.1 * dW1
    b1 -= 0.1 * db1
hidden = sigmoid(np.dot(X_test, W1) + b1)
output = sigmoid(np.dot(hidden, W2) + b2)
pred = (output >= 0.5).astype(int)
print("Artificial Neural Network")
print("Training Samples:", len(X_train))
print("Testing Samples:", len(X_test))
print("Predicted Output:", data.target_names[pred[0][0]])
print("Actual Output:", data.target_names[y_test[0][0]])
print("Accuracy:", round(accuracy_score(y_test, pred) * 100, 2), "%")
```

OUTPUT:

```
print("Training Samples:", len(X_train))
print("Testing Samples:", len(X_test))
print("Predicted Output:", data.target_names[pred[0][0]])
print("Actual Output:", data.target_names[y_test[0][0]])
print("Accuracy:", round(accuracy_score(y_test, pred) * 100, 2), "%")
```

```
Artificial Neural Network
Training Samples: 455
Testing Samples: 114
Predicted Output: benign
Actual Output: benign
Accuracy: 99.12 %
```